

Filtering_



Contents

1 0. 오리엔테이션

1.1 본 자료의 구성

1.2 영상 필터링이란

2 1. 평균값 필터 (Mean Filter)

2.1 평균값 필터의 마스크

2.2 마스크가 커질수록 더 부드러워짐

2.3 cv2.filter2D() — 직접 만든 마스크 적용

2.4 실습 코드 — 5×5 평균값 필터

2.5 코드 들여다보기 — 사용한 함수·클래스 (1)

2.6 실행 결과 — filter2D vs blur

2.7 cv2.blur() — 평균값 필터 전용 함수

2.8 실습 코드 — 마스크 크기 3·5·7 비교

2.9 코드 들여다보기 — 사용한 함수·클래스 (2)

2.10 실행 결과 — 커널 크기별 흐림 정도

3 2. 가우시안 필터 (Gaussian Filter)

3.1 평균값 필터와의 차이 — 가중치

3.2 가우시안 함수의 모양

3.3 2차원 공간에서의 가우시안

3.4 가우시안 마스크의 실제 계수 예

3.5 cv2.GaussianBlur() — 가우시안 블러 함수

3.6 실습 코드 — sigma 1~5 비교

3.7 코드 들여다보기 — 사용한 함수·클래스 (3)

3.8 실행 결과 — sigma에 따른 흐림 변화

4 3. 샤프닝 (Sharpening)

4.1 언샤프 마스크 필터 — 개념

4.2 실습 코드 — 흑백 영상 샤프닝

4.3 코드 들여다보기 — 사용한 함수·클래스 (4)

4.4 실행 결과 — 흑백 샤프닝

4.5 컬러 영상 샤프닝 — 밝기 채널만 처리

4.6 코드 들여다보기 — 사용한 함수·클래스 (5)

4.7 실행 결과 — 컬러 샤프닝

5 4. 잡음 제거 (Noise Reduction)

5.1 영상 잡음의 종류

5.2 메디안 필터의 원리

5.3 cv2.medianBlur() — 메디안 필터 함수

5.4 실습 코드 — 메디안 필터

5.5 코드 들여다보기 — 사용한 함수·클래스 (6)

5.6 실행 결과 — 메디안 필터

5.7 가우시안 필터는 영상 전체를 blurring

5.8 양방향 필터는 에지를 뺀 부분만 blurring

5.9 cv2.bilateralFilter() — 양방향 필터 함수

5.10 실습 코드 — 양방향 필터

5.11 코드 들여다보기 — 사용한 함수·클래스 (7)

5.12 실행 결과 — 양방향 필터

6 5. 재미있는 예제 — 카툰·스케치 필터

6.1 카툰 필터의 구성

6.2 스케치 필터의 구성

6.3 실습 코드 — 필터 함수 정의

6.4 코드 들여다보기 — 사용한 함수·클래스 (8)

6.5 실습 코드 — 카메라 실시간 적용

6.6 코드 들여다보기 — 사용한 함수·클래스 (9)

7 6. 정리

7.1 필터 비교표

7.2 파라미터가 커지면 어떻게 되나

7.3 학습 포인트

Filtering_

Exported from Confluence · <https://pinkwink.atlassian.net>

0. 오리엔테이션

본 자료의 구성

영상 필터링 — 학습 흐름



학습 순서

- 평균값 필터 → 가우시안 필터 → 샤프닝 → 잡음 제거 → 응용 필터 순으로 진행
- 각 필터마다 원리(그림) → 함수 시그니처 → 실습 코드 → 실행 결과 4단계 반복
- 실습 영상: rose.bmp(부드럽게·샤프닝), noise.bmp(잡음 제거), lenna.bmp(양방향 필터)

영상 필터링이란

핵심 개념

- **필터링(filtering)**: 영상에서 원하는 정보만 통과시키고 나머지는 걸러 내는 연산
- **마스크(mask)·커널(kernel)**: 필터의 모양·계수를 담은 작은 행렬. 보통 3×3, 5×5 등 홀수 크기 정사각형
- **합성곱(convolution)**: 마스크를 영상 위에서 한 픽셀씩 이동시키며 겹친 값끼리 곱해 더한 결과를 중앙 픽셀에 기록
- **고정점(anchor)**: 결과를 저장할 기준 위치. 보통 마스크 중앙
- **가장자리 처리(borderType)**: 마스크가 영상 밖으로 나가는 구간을 어떻게 채울지 결정

필터의 두 갈래

- **블러(저역 통과)**: 인접 픽셀 값을 섞어 부드럽게 함 → 평균값·가우시안·메디안·양방향 필터
- **샤프닝(고역 강조)**: 부드러운 성분을 빼서 경계를 도드라지게 함 → 언샤프 마스크

1. 평균값 필터 (Mean Filter)

평균값 필터의 마스크

$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} \qquad \frac{1}{25} \times \begin{array}{|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline \end{array}$$

원리

- 마스크 안 모든 픽셀에 **같은 가중치**를 주고 산술 평균을 계산
- 3×3 마스크는 계수 1을 9개 두고 1/9배, 5×5는 25개 두고 1/25배 → 계수 합이 항상 1
- 계수 합이 1이어야 원본 밝기가 유지됨(합이 1보다 크면 밝아지고 작으면 어두워짐)
- 잡음은 줄지만 **경계(에지)도 함께 뭉개짐**

마스크가 커질수록 더 부드러워짐



원본 영상



3x3 크기의 마스크



5x5 크기의 마스크

- 3×3 → 5×5로 커질수록 평균에 참여하는 픽셀이 늘어 흐림 정도가 강해짐
- 마스크가 커질수록 연산량도 함께 증가 → 목적에 맞는 최소 크기 선택이 중요

cv2.filter2D() — 직접 만든 마스크 적용

```
cv2.filter2D(src, ddepth, kernel, dst=None, anchor=None, delta=None,
             borderType=None) -> dst
```

- src: 입력 영상
- ddepth: 출력 영상 데이터 타입. (e.g) cv2.CV_8U, cv2.CV_32F, cv2.CV_64F
-1을 지정하면 src와 같은 타입의 dst 영상을 생성
- kernel: 필터 마스크 행렬. 실수형.
- anchor: 고정점 위치. (-1, -1)이면 필터 중앙을 고정점으로 사용
- delta: 추가적으로 더할 값
- borderType: 가장자리 픽셀 확장 방식
- dst: 출력 영상

사용 포인트

- 임의의 커널을 직접 설계해 적용할 때 사용하는 범용 함수
- `ddepth=-1` 이면 입력과 같은 자료형으로 출력
- `anchor=(-1, -1)` (기본)이면 마스크 중앙이 고정점

실습 코드 — 5×5 평균값 필터

py

```
1 import sys
2 import cv2
3 import numpy as np
4
5 src = cv2.imread('./data/rose.bmp', cv2.IMREAD_GRAYSCALE)
6
7 if src is None:
8     print('Image load failed!')
9     sys.exit()
10
11 kernel = np.ones(shape=(5, 5), dtype=np.float32) / 25
12
13 dst1 = cv2.filter2D(src, -1, kernel)
14 dst2 = cv2.blur(src, (3, 3))
15
16 cv2.imshow('src', src)
17 cv2.imshow('dst1', dst1)
18 cv2.imshow('dst2', dst2)
19 cv2.waitKey()
20
21 cv2.destroyAllWindows()
```

- 실습 데이터: https://drive.google.com/file/d/1TmW-rQKpGc_S7D956wFmyiMgCXN41R4X/view?usp=sharing (https://drive.google.com/file/d/1TmW-rQKpGc_S7D956wFmyiMgCXN41R4X/view?usp=sharing)

코드 들여다보기 — 사용한 함수·클래스 (1)

이름	종류	설명
sys	모듈	인터프리터 제어 표준 모듈. 여기서 <code>sys.exit()</code> 로 조기 종료
cv2	모듈	OpenCV 파이썬 바인딩
numpy (np)	모듈	행렬 연산 라이브러리. 커널 생성에 사용
cv2.imread()	함수	영상 파일을 읽어 ndarray로 반환. 실패 시 None
cv2.IMREAD_GRAYSCALE	상수	그레이스케일 1채널로 읽으라는 플래그
sys.exit()	함수	파일 로드 실패 시 프로그램 즉시 종료

np.ones()	함수	지정한 shape을 1로 채운 배열 생성. 25로 나눠 평균 마스크 완성
np.float32	자료형	커널 계수용 32비트 실수형(필터 마스크는 실수형이어야 함)
cv2.filter2D()	함수	사용자 정의 커널로 합성곱 수행
cv2.blur()	함수	지정 크기의 평균값 필터를 한 번에 적용
cv2.imshow()	함수	창을 만들어 영상 출력
cv2.waitKey()	함수	키 입력 대기(인자 없으면 무한 대기)
cv2.destroyAllWindows()	함수	열린 모든 창 닫기

실행 결과 — filter2D vs blur



- `dst1` (5×5 직접 커널)이 `dst2` (3×3 blur)보다 더 흐림
- 두 방식의 결과는 동일 크기의 커널이라면 사실상 같음 → 평균 필터는 `blur()`가 간편

cv2.blur() — 평균값 필터 전용 함수

```
cv2.blur(src, ksize, dst=None, anchor=None, borderType=None) -> dst
```

- src: 입력 영상
- ksize: 평균값 필터 크기. (width, height) 형태의 튜플.
- dst: 결과 영상. 입력 영상과 같은 크기 & 같은 타입.

$$\text{kernel} = \frac{1}{\text{ksize.width} \times \text{ksize.height}} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

사용 포인트

- `ksize` 는 (width, height) 튜플 → 내부에서 $1/(w \times h)$ 계수 커널을 자동 생성
- 커널을 직접 만들 필요가 없어 코드가 짧아짐

실습 코드 — 마스크 크기 3·5·7 비교

py

```
1 import sys
2 import numpy as np
3 import cv2
4
5 src = cv2.imread('./data/rose.bmp', cv2.IMREAD_GRAYSCALE)
6
7 if src is None:
8     print('Image load failed!')
9     sys.exit()
10
11 cv2.imshow('src', src)
12
13 for ksize in (3, 5, 7):
14     dst = cv2.blur(src, (ksize, ksize))
15
16     desc = 'Mean: {}x{}'.format(ksize, ksize)
17     cv2.putText(dst, desc, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
18                 1.0, 255, 1, cv2.LINE_AA)
19
20     cv2.imshow('dst', dst)
21
22 cv2.waitKey()
23 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (2)

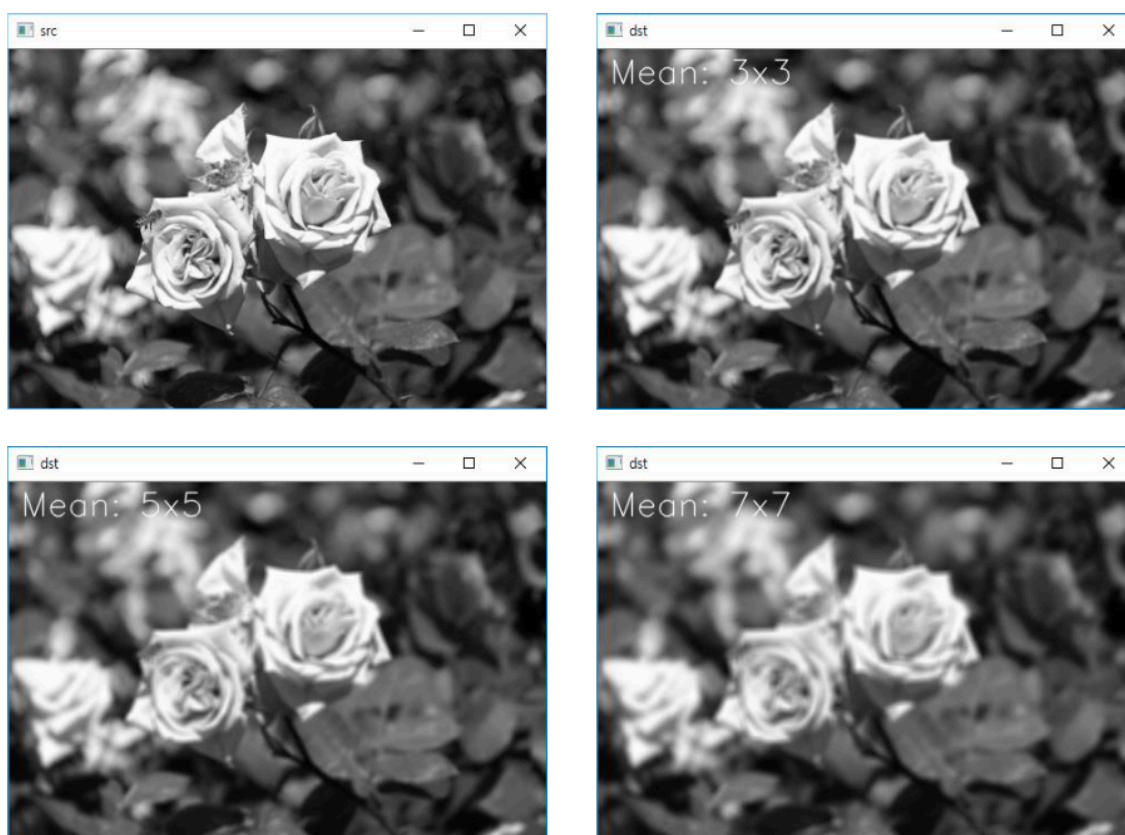
이름	종류	설명
for 반복	구문	커널 크기 3·5·7을 순회하며 같은 영상에 반복 적용
cv2.blur()	함수	(ksize, ksize) 크기 평균값 필터 적용
str.format()	메서드	커널 크기를 문자열로 삽입해 라벨 생성
cv2.putText()	함수	영상 위에 문자열을 그려 넣음(현재 커널 크기 표시)
cv2.FONT_HERSHEY_SIMPLEX	상수	기본 산세리프 폰트 지정
cv2.LINE_AA	상수	안티에일리어싱 선 타입 → 글자 경계가 부드러움

cv2.waitKey()	함수	반복문 밖에서 한 번 대기 → 마지막 결과 창 유지
---------------	----	------------------------------

실습 팁

- `cv2.imshow('dst', dst)` 가 같은 창 이름을 재사용 → 반복 중에는 마지막 결과만 보임
- 단계별로 확인하려면 반복문 안에 `cv2.waitKey()` 를 넣거나 창 이름을 `'dst'+str(ksize)` 처럼 다르게 지정

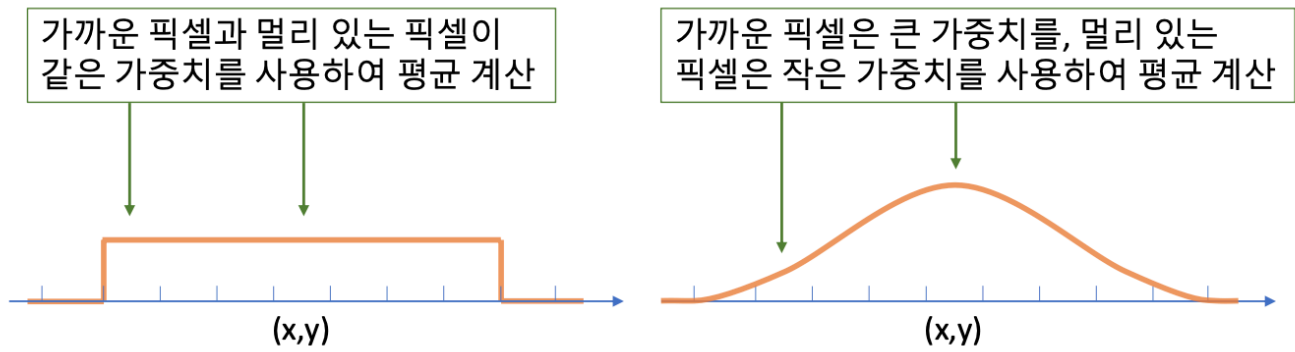
실행 결과 — 커널 크기별 흐림 정도



- 커널이 커질수록 잡음은 잘 지워지지만 꽃잎 경계까지 흐려짐
- 흐림 정도와 디테일 보존은 트레이드오프

2. 가우시안 필터 (Gaussian Filter)

평균값 필터와의 차이 — 가중치



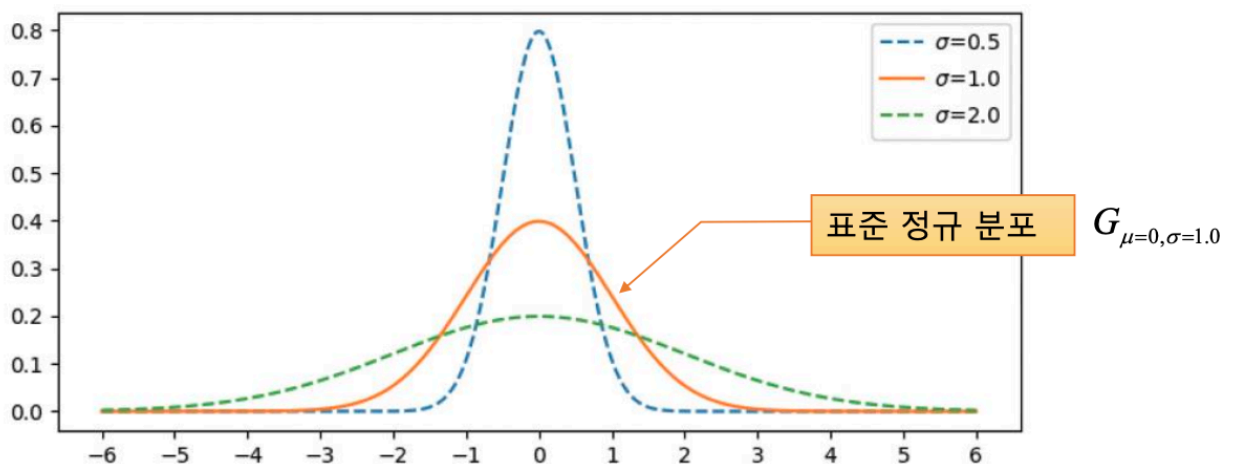
핵심 차이

- **평균값 필터**: 가까운 픽셀과 먼 픽셀에 **같은 가중치** → 사각형(박스) 모양 가중치
- **가우시안 필터**: 가까운 픽셀에 큰 가중치, 먼 픽셀에 작은 가중치 → 종 모양 가중치
- 결과적으로 가우시안 쪽이 더 자연스러운 블러를 만들고 격자 형태의 인공적 흔적이 적음

가우시안 함수의 모양

$$G_{\mu,\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- μ : 평균
- σ : 표준편차

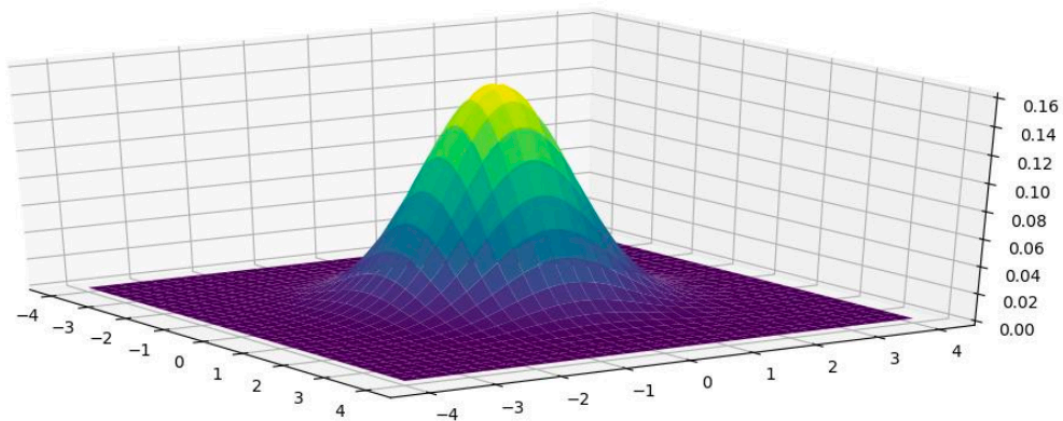


수식 읽기

- 평균 μ 가 분포의 중심, 표준편차 σ 가 퍼짐 정도를 결정
- 필터에서는 중앙 픽셀이 기준이므로 $\mu = 0$ 으로 두고 σ 만 조절
- σ 가 클수록 넓게 퍼짐 → 더 강한 블러

2차원 공간에서의 가우시안

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{\left(-\frac{x^2+y^2}{2\sigma^2}\right)} \quad \begin{cases} \mu_x = \mu_y = 0 \\ \sigma_x = \sigma_y = \sigma \end{cases}$$



- 영상은 2차원이므로 x·y 두 방향의 가우시안을 곱한 형태를 사용
- 2차원 가우시안은 x방향 1차원 × y방향 1차원으로 분리 계산 가능 → 연산량 절감(분리 가능 필터)

가우시안 마스크의 실제 계수 예

4	0.0000	0.0000	0.0000	0.0000	0.0001	0.0000	0.0000	0.0000	0.0000
3	0.0000	0.0000	0.0002	0.0011	0.0018	0.0011	0.0002	0.0000	0.0000
2	0.0000	0.0002	0.0029	0.0131	0.0215	0.0131	0.0029	0.0002	0.0000
1	0.0000	0.0011	0.0131	0.0585	0.0965	0.0585	0.0131	0.0011	0.0000
0	0.0001	0.0018	0.0215	0.0965	0.1592	0.0965	0.0215	0.0018	0.0001
-1	0.0000	0.0011	0.0131	0.0585	0.0965	0.0585	0.0131	0.0011	0.0000
-2	0.0000	0.0002	0.0029	0.0131	0.0215	0.0131	0.0029	0.0002	0.0000
-3	0.0000	0.0000	0.0002	0.0011	0.0018	0.0011	0.0002	0.0000	0.0000
-4	0.0000	0.0000	0.0000	0.0000	0.0001	0.0000	0.0000	0.0000	0.0000
	-4	-3	-2	-1	0	1	2	3	4

- 중앙 계수가 가장 크고(0.1592) 바깥으로 갈수록 급격히 0에 수렴
- $\sigma \approx 1.0$ 이면 중앙에서 약 $\pm 3\sigma$ ($\approx \pm 3$ 픽셀) 밖은 사실상 0 $\rightarrow$ 커널 크기를 σ 로부터 자동 결정 가능

cv2.GaussianBlur() — 가우시안 블러 함수

```
cv2.GaussianBlur(src, ksize, sigmaX, dst=None, sigmaY=None,
                 borderType=None) -> dst
```

- src: 입력 영상. 각 채널 별로 처리됨.
- dst: 출력 영상. src와 같은 크기, 같은 타입.
- ksize: 가우시안 커널 크기. (0, 0)을 지정하면 sigma 값에 의해 자동 결정됨.
- sigmaX: x방향 sigma.
- sigmaY: y방향 sigma. 0이면 sigmaX와 같게 설정.
- borderType: 가장자리 픽셀 확장 방식.

사용 포인트

- `ksize=(0, 0)` 으로 두고 **sigmaX**만 지정하는 방식이 일반적 → 커널 크기 자동 계산
- **sigmaY** 를 0으로 두면 **sigmaX** 와 동일하게 설정
- 컬러 영상은 각 채널별로 독립 처리

실습 코드 — sigma 1~5 비교

py

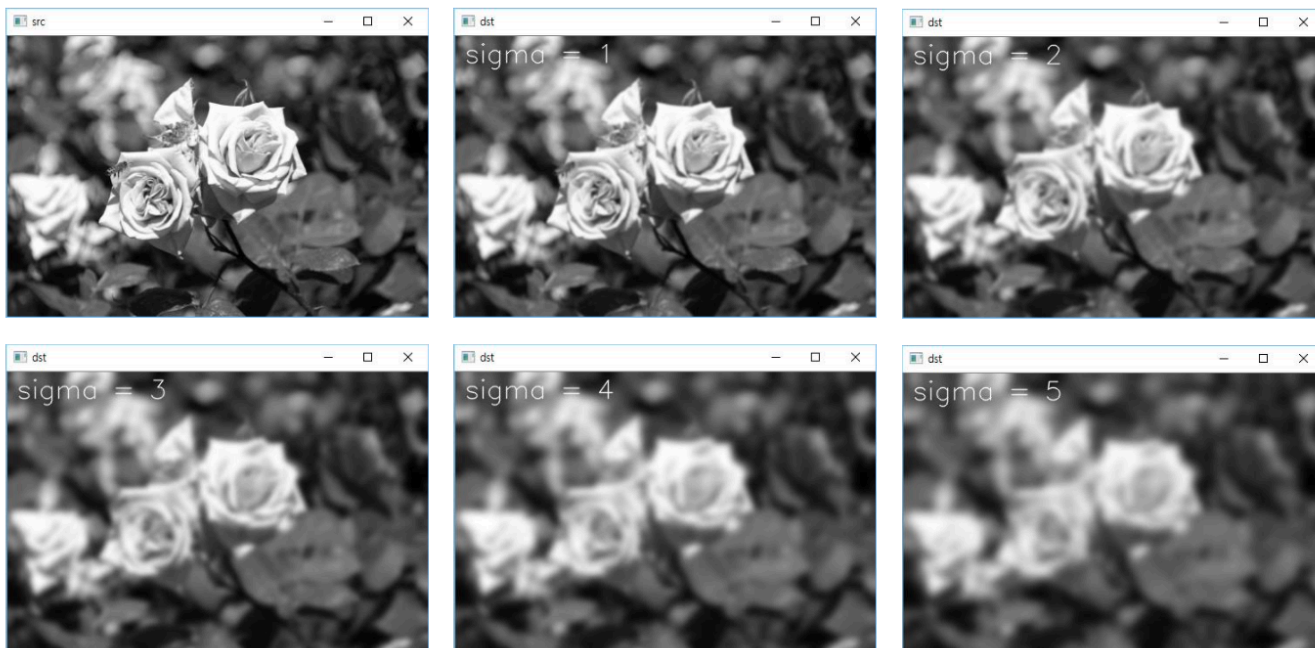
```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread('./data/rose.bmp', cv2.IMREAD_GRAYSCALE)
6
7  if src is None:
8      print('Image load failed!')
9      sys.exit()
10
11  cv2.imshow('src', src)
12
13  for sigma in range(1, 6):
14      # sigma 값을 이용하여 가우시안 필터링
15      dst = cv2.GaussianBlur(src, (0, 0), sigma)
16
17      desc = 'sigma = {}'.format(sigma)
18      cv2.putText(dst, desc, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
19                  1.0, 255, 1, cv2.LINE_AA)
20
21      cv2.imshow('dst', dst)
22      cv2.waitKey()
23
24  cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (3)

이름	종류	설명
<code>range()</code>	함수	1부터 5까지 sigma 값 생성
<code>cv2.GaussianBlur()</code>	함수	가우시안 블러 수행. <code>ksize=(0,0)</code> → sigma로 커널 크기 자동 결정

cv2.putText()	함수	현재 sigma 값을 영상에 표시
cv2.waitKey()	함수	반복문 안에서 대기 → sigma별 결과를 하나씩 확인 가능
cv2.destroyAllWindows()	함수	반복 종료 후 모든 창 닫기

실행 결과 — sigma에 따른 흐림 변화



- sigma가 커질수록 흐림이 급격히 강해짐
- 실무에서는 커널 크기 대신 sigma로 흐림 강도를 조절하는 편이 직관적

3. 샤프닝 (Sharpening)

언샤프 마스크 필터 — 개념

핵심 개념

- 날카로운 성분 = 원본 - 부드러운 성분 이라는 관계를 이용
- 블러 영상(`blr`)을 만들고 원본에서 빼면 경계 성분만 남음
- 이를 원본에 다시 더함 → $dst = src + (src - blr) = 2 \cdot src - blr$
- 일반화하면 $dst = src + \alpha \cdot (src - blr)$, α 가 클수록 더 강한 샤프닝
- 블러의 sigma가 클수록 굵은 경계가, 작을수록 미세한 디테일이 강조됨

샤프닝 결과는 0~255 범위를 벗어날 수 있음 → 반드시 `np.clip()` 으로 자르고 `uint8` 로 되돌릴 것. 실수형 연산을 위해 `float32` 로 변환 후 계산하는 것이 안전함.

실습 코드 — 흑백 영상 샤프닝

py

```
1 import sys
2 import numpy as np
3 import cv2
4
5 src = cv2.imread('./data/rose.bmp', cv2.IMREAD_GRAYSCALE)
6
7 if src is None:
8     print('Image load failed!')
9     sys.exit()
10
11 blr = cv2.GaussianBlur(src, (0, 0), 2)
12 dst = np.clip(2.0*src - blr, 0, 255).astype(np.uint8)
13
14 cv2.imshow('src', src)
15 cv2.imshow('blr', blr)
16 cv2.imshow('dst', dst)
17 cv2.waitKey()
18
19 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (4)

이름	종류	설명
----	----	----

cv2.GaussianBlur()	함수	sigma=2의 부드러운 영상 <code>blr</code> 생성
2.0*src - blr	연산	실수 <code>2.0</code> 을 곱해 넘파이 배열이 float로 자동 승격 → 오버플로 방지
np.clip()	함수	0 미만·255 초과 값을 잘라 유효 화소 범위로 제한
.astype()	메서드	실수 결과를 <code>np.uint8</code> 로 변환해 영상 자료형 복원
np.uint8	자료형	0~255 8비트 부호 없는 정수 — 일반 영상 화소 자료형

실행 결과 — 흑백 샤프닝



- `blr` 는 뭉개진 영상, `dst` 는 꽃잎 경계가 또렷해진 영상
- 샤프닝은 **잡음도 함께 강조**하므로 잡음이 심한 영상엔 잡음 제거를 먼저 적용

컬러 영상 샤프닝 — 밝기 채널만 처리

왜 YCrCb인가

- BGR 세 채널을 각각 샤프닝하면 **색상이 틀어짐**
- YCrCb로 변환하면 밝기(Y)와 색상(Cr, Cb)이 분리됨
- **Y 채널만 샤프닝**한 뒤 다시 BGR로 되돌리면 색 왜곡 없이 선명도만 개선

py

```

1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread("./data/rose.bmp")
6
7  if src is None:
8      print("Image load failed!")
9      sys.exit()
10
11  src_ycrb = cv2.cvtColor(src, cv2.COLOR_BGR2YCrCb)
12
13  src_f = src_ycrb[:, :, 0].astype(np.float32)
14  blr = cv2.GaussianBlur(src_f, (0, 0), 2.0)          # 밝기 채널만 블러
15  src_ycrb[:, :, 0] = np.clip(2.0 * src_f - blr, 0, 255).astype(np.uint8)
16
17  dst = cv2.cvtColor(src_ycrb, cv2.COLOR_YCrCb2BGR)
18
19  cv2.imshow("src", src)
20  cv2.imshow("dst", dst)
21  cv2.waitKey()
22
23  cv2.destroyAllWindows()

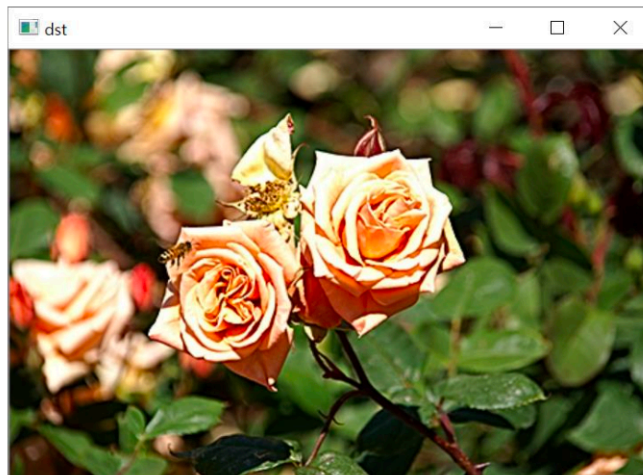
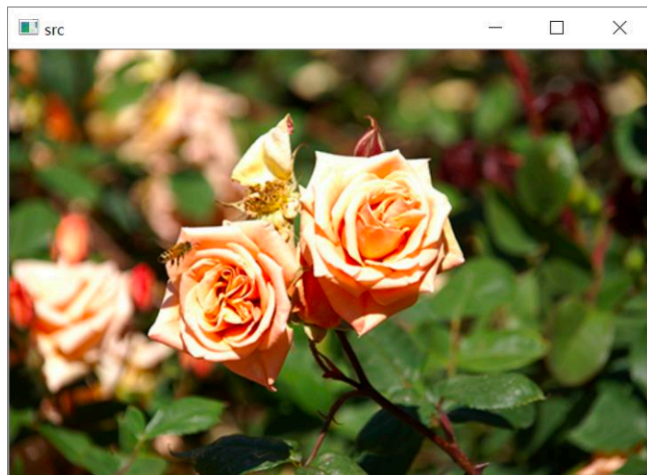
```

원본 화면 캡처에는 `blr` 을 만드는 줄이 보이지 않음. `blr = cv2.GaussianBlur(src_f, (0, 0), 2.0)` 한 줄이 반드시 있어야 실행됨(위 코드에 포함).

코드 들여다보기 — 사용한 함수·클래스 (5)

이름	종류	설명
<code>cv2.imread()</code>	함수	플래그 없이 호출 → BGR 3채널 컬러로 읽음
<code>cv2.cvtColor()</code>	함수	색 공간 변환 수행
<code>cv2.COLOR_BGR2YCrCb</code>	상수	BGR → YCrCb 변환 플래그(밝기·색상 분리)
<code>cv2.COLOR_YCrCb2BGR</code>	상수	처리 후 다시 BGR로 되돌리는 플래그
<code>src_ycrCb[:, :, 0]</code>	슬라이싱	0번 채널(Y, 밝기)만 선택
<code>.astype(np.float32)</code>	메서드	밝기 채널을 실수형으로 변환해 뺄셈 오차·오버플로 방지
<code>np.clip()</code>	함수	0~255로 범위 제한 후 uint8로 복원해 채널에 다시 대입

실행 결과 — 컬러 샤프닝



- 색 번짐 없이 윤곽만 선명해짐
- α (위 코드의 계수 2.0)를 키우면 더 강해지지만 **경계 주변에 흰/검은 테두리(halo)**가 생길 수 있음

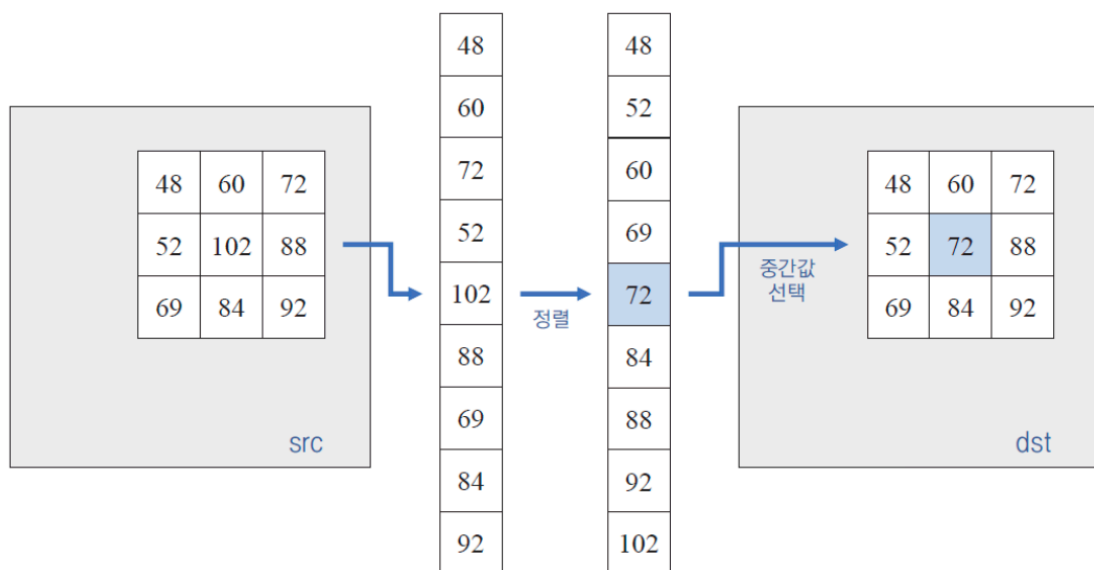
4. 잡음 제거 (Noise Reduction)

영상 잡음의 종류

개념 정리

- 가우시안 잡음: 화소값에 정규분포 형태의 오차가 더해짐 → 전체가 자글자글
- 소금·후추 잡음(임펄스): 일부 화소가 0 또는 255로 됨 → 흰·검은 점
- 평균·가우시안 필터는 튀는 값도 평균에 섞어 버려 **임펄스 잡음에 약함**
- 임펄스 잡음엔 **메디안 필터**, 경계를 살리며 부드럽게 하려면 **양방향 필터**가 적합

메디안 필터의 원리



동작 방식

- 마스크 내 픽셀값을 정렬한 뒤 **가운데 값(중간값)**을 선택
- 예: 3×3 안의 102(튀는 값)가 정렬 후 가장자리로 밀려나고 중앙값 72가 선택됨
- 곱셈·덧셈이 아닌 **순서 통계** 기반 → 비선형 필터
- 튀는 값을 계산에 섞지 않으므로 **경계는 비교적 유지하면서 임펄스 잡음만 제거**

`cv2.medianBlur()` — 메디안 필터 함수

```
cv2.medianBlur(src, ksize, dst=None) -> dst
```

- src: 입력 영상. 각 채널 별로 처리됨.
- ksize: 커널 크기. 1보다 큰 홀수를 지정.
- dst: 출력 영상. src와 같은 크기, 같은 타입.

사용 포인트

- `ksize` 는 1보다 큰 홀수만 가능(3, 5, 7 ...) → 중간값을 정하려면 홀수 개여야 함
- 커널 크기가 커질수록 강력하지만 영상이 뭉툭해짐(붓으로 칠한 느낌)

실습 코드 — 메디안 필터

py

```
1 import sys
2 import cv2
3
4 src = cv2.imread('./data/noise.bmp', cv2.IMREAD_GRAYSCALE)
5
6 if src is None:
7     print('Image load failed!')
8     sys.exit()
9
10 dst = cv2.medianBlur(src, 3)
11
12 cv2.imshow('src', src)
13 cv2.imshow('dst', dst)
14 cv2.waitKey()
15
16 cv2.destroyAllWindows()
```

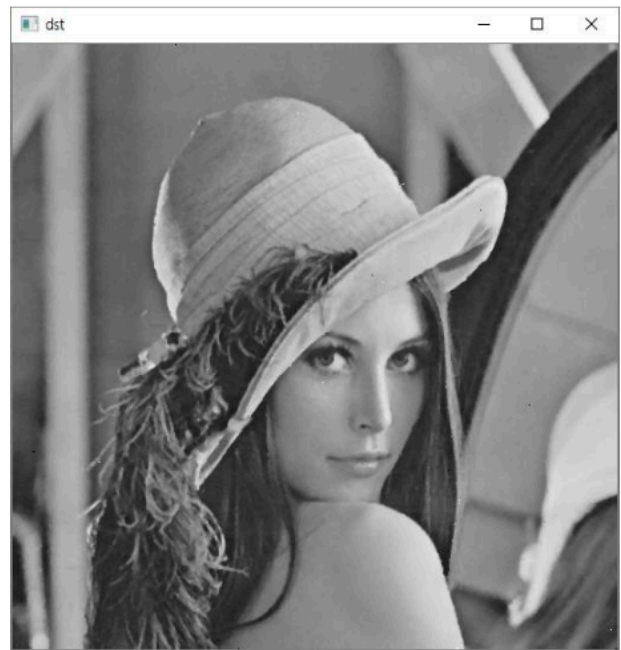
- 실습 데이터: https://drive.google.com/file/d/14W7Ef_8COLsdpWkv9FEm4ksVaUp2_LR1/view?usp=sharing (https://drive.google.com/file/d/14W7Ef_8COLsdpWkv9FEm4ksVaUp2_LR1/view?usp=sharing)

코드 들여다보기 — 사용한 함수·클래스 (6)

이름	종류	설명
----	----	----

cv2.imread()	함수	잡음이 섞인 noise.bmp를 그레이스케일로 읽음
cv2.medianBlur()	함수	3×3 메디안 필터 적용 — 소금·후추 잡음 제거
cv2.imshow() / waitKey() / destroyAllWindows()	함수	원본·결과 비교 출력 후 키 입력 대기, 창 정리

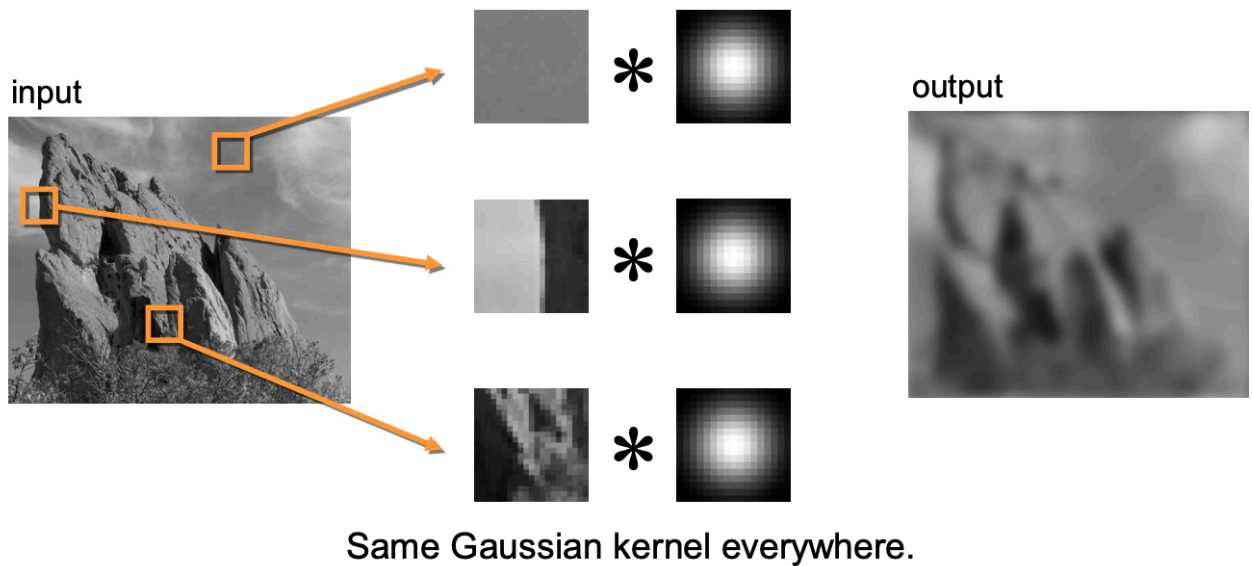
실행 결과 — 메디안 필터



- 점 형태의 잡음이 거의 사라지고 글자·윤곽은 유지됨
- 같은 영상에 평균값 필터를 쓰면 점이 번져서 회색 얼룩이 됨 → 잡음 종류에 맞는 필터 선택이 핵심

가우시안 필터는 영상 전체를 blurring

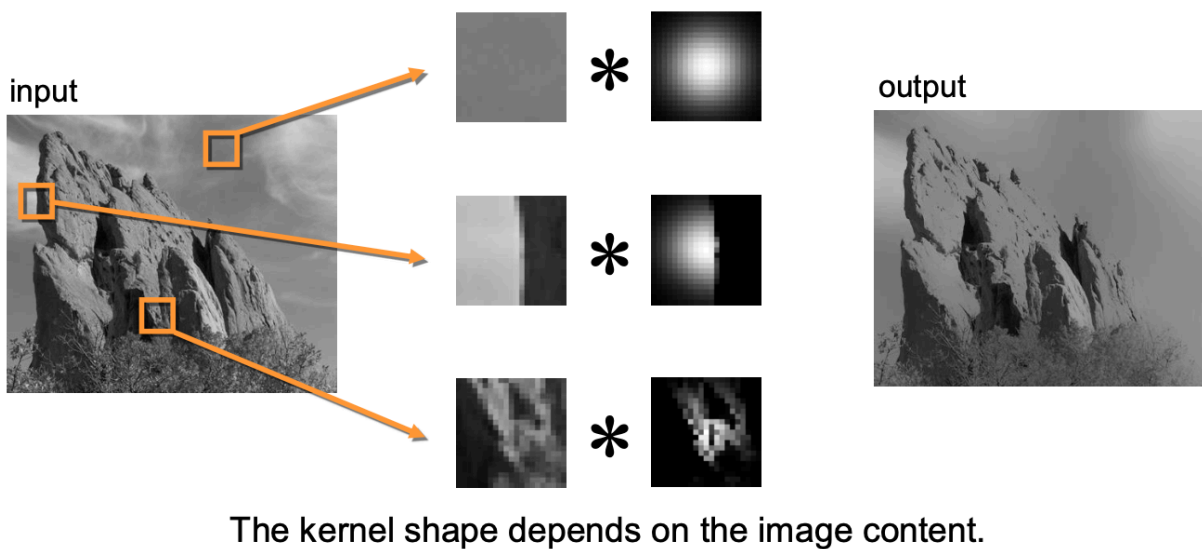
■ (일반적인) 가우시안 필터링: 영상 전체에서 blurring



- 위치와 무관하게 **항상 같은 모양의 커널**을 사용(Same Gaussian kernel everywhere)
- 경계 부분에서도 양쪽 값을 평균 → 경계가 함께 뭉개짐

양방향 필터는 에지를 뺀 부분만 blurring

■ 양방향 필터: 에지가 아닌 부분에서만 blurring



핵심 개념

- **양방향(bilateral) 필터**: 공간 거리 가중치 × 색상(밝기) 차이 가중치를 함께 사용
- 밝기 차이가 큰 이웃(= 경계 건너편)은 가중치가 거의 0 → 평균에 끼지 못함
- 결과적으로 커널 모양이 **영상 내용에 따라 달라짐**(The kernel shape depends on the image content)
- 에지는 보존하면서 평탄한 영역만 부드럽게 → 피부 보정·카툰 효과의 기본기
- 단점: 연산량이 큼(가우시안 필터보다 훨씬 느림)

cv2.bilateralFilter() — 양방향 필터 함수

```
cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace, dst=None,
                    borderType=None) -> dst
```

- src: 입력 영상. 8비트 또는 실수형, 1채널 또는 3채널.
- d: 필터링에 사용될 이웃 픽셀의 거리(지름).
음수(-1)를 입력하면 sigmaSpace 값에 의해 자동 결정됨.
- sigmaColor: 색 공간에서 필터의 표준 편차
- sigmaSpace: 좌표 공간에서 필터의 표준 편차
- dst: 출력 영상. src와 같은 크기, 같은 타입.
- borderType: 가장자리 픽셀 처리 방식

파라미터 감각

- **d**: 이웃 픽셀 지름. -1이면 sigmaSpace로 자동 결정(권장)
- **sigmaColor**: 클수록 더 많은 밝기 차이를 '같은 영역'으로 간주 → 에지도 뭉개짐
- **sigmaSpace**: 클수록 넓은 범위를 참조 → 느려짐
- 실무 감각: sigmaColor는 10~50, sigmaSpace는 5~10에서 시작해 조절

실습 코드 — 양방향 필터

py

```
1 import sys
2 import numpy as np
3 import cv2
4
5 src = cv2.imread('./data/lenna.bmp')
6
7 if src is None:
8     print('Image load failed!')
9     sys.exit()
10
11 dst = cv2.bilateralFilter(src, -1, 10, 5)
12
13 cv2.imshow('src', src)
14 cv2.imshow('dst', dst)
15 cv2.waitKey()
16
17 cv2.destroyAllWindows()
```

- 실습 데이터: https://drive.google.com/file/d/1Pdk7jIWVpbUcSoZQcc9x_CXlu0VeMwf_/view?usp=sharing
(https://drive.google.com/file/d/1Pdk7jIWVpbUcSoZQcc9x_CXlu0VeMwf_/view?usp=sharing)

코드 들여다보기 — 사용한 함수·클래스 (7)

이름	종류	설명
cv2.imread()	함수	lenna.bmp를 컬러(BGR)로 읽음
cv2.bilateralFilter()	함수	양방향 필터 적용 — d=-1(자동), sigmaColor=10, sigmaSpace=5
d = -1	인자	이웃 지름을 sigmaSpace 기반으로 자동 계산
sigmaColor = 10	인자	밝기 차 10 정도까지만 같은 영역으로 취급 → 에지 보존
sigmaSpace = 5	인자	참조하는 공간 범위. 값이 커지면 계산이 크게 느려짐

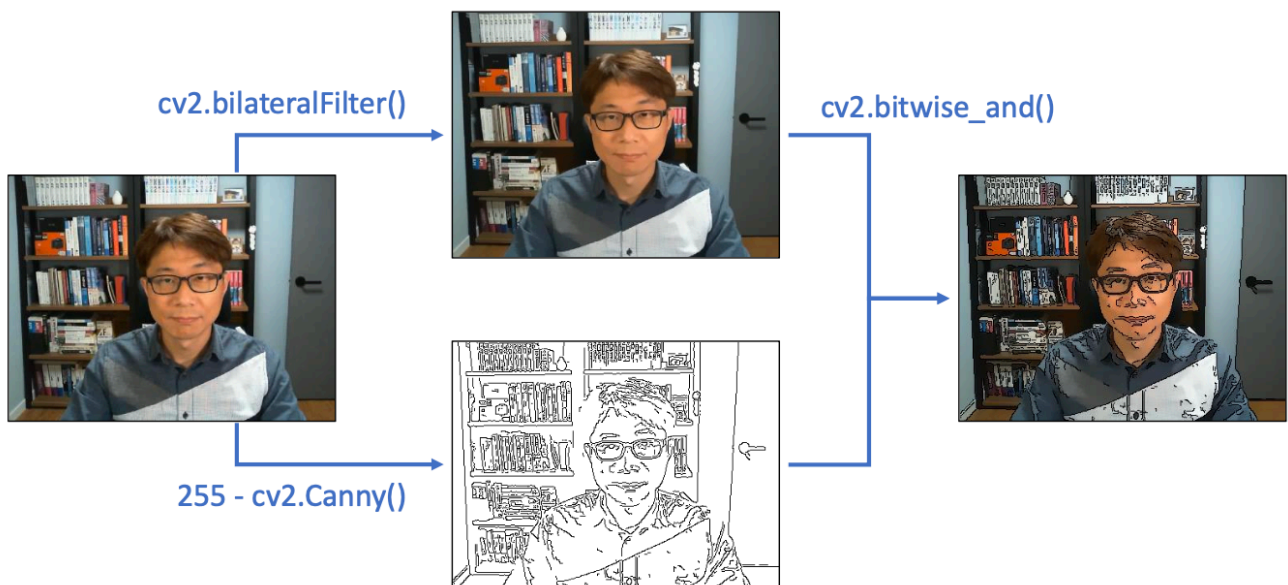
실행 결과 — 양방향 필터



- 피부·배경 같은 평탄한 영역은 매끈해지고 모자 테두리·눈매 등 경계는 살아 있음
- 가우시안 블러 결과와 비교하면 차이가 뚜렷 → **에지 보존 스무딩**의 대표 사례

5. 재미있는 예제 — 카툰·스케치 필터

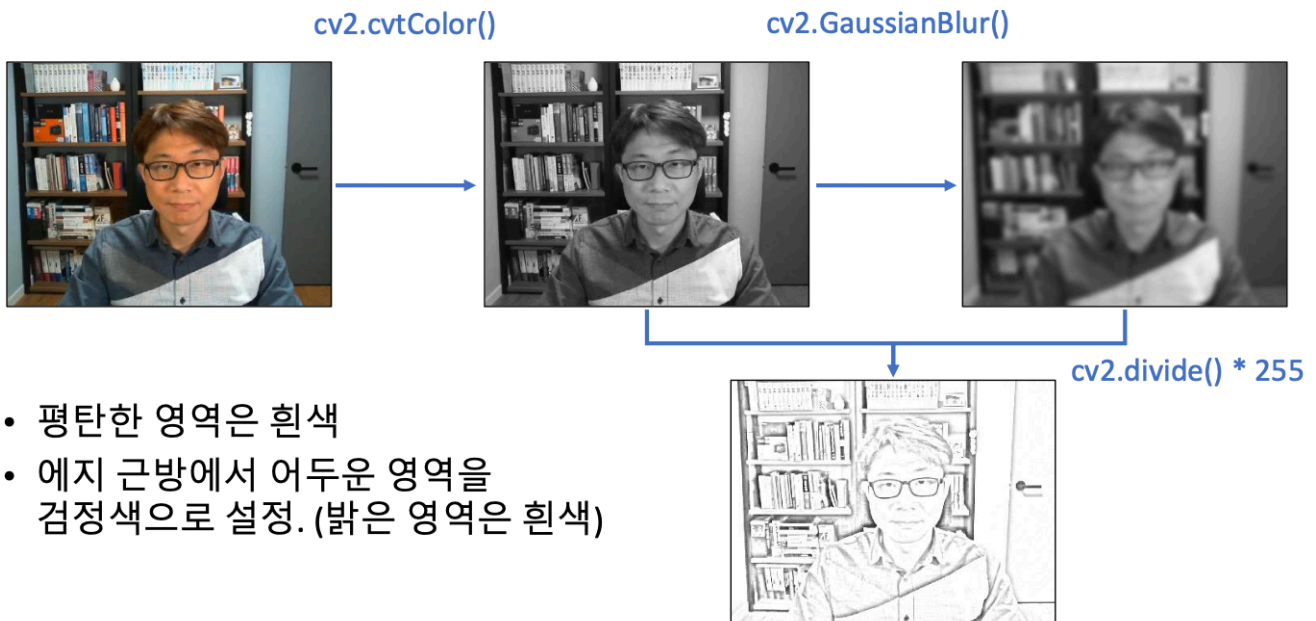
카툰 필터의 구성



처리 흐름

- ① `cv2.bilateralFilter()` → 색을 뭉쳐 만화처럼 평탄한 색면 생성
- ② `255 - cv2.Canny()` → 에지를 검은 선, 배경을 흰색으로 반전
- ③ `cv2.bitwise_and()` → 색면 위에 검은 윤곽선을 합성
- 속도를 위해 처리 전 영상을 절반으로 축소하고 결과를 다시 확대

스케치 필터의 구성



처리 흐름

- ① `cv2.cvtColor()` → 그레이스케일 변환
- ② `cv2.GaussianBlur()` → 흐린 영상 생성
- ③ `cv2.divide(gray, blr, scale=255)` → 원본 ÷ 흐린 영상
- 평탄한 영역은 두 값이 비슷해 나눗셈 결과가 1 → 255(흰색)
- 에지 근방은 원본이 상대적으로 어두워 결과가 작아짐 → 검은 선

실습 코드 — 필터 함수 정의

py

```
1  # 카툰 필터 카메라
2
3  import sys
4  import numpy as np
5  import cv2
6
7
8  def cartoon_filter(img):
9      h, w = img.shape[:2]
10     img2 = cv2.resize(img, (w//2, h//2))
11
12     blr = cv2.bilateralFilter(img2, -1, 20, 7)
13     edge = 255 - cv2.Canny(img2, 80, 120)
14     edge = cv2.cvtColor(edge, cv2.COLOR_GRAY2BGR)
15
16     dst = cv2.bitwise_and(blr, edge)
17     dst = cv2.resize(dst, (w, h), interpolation=cv2.INTER_NEAREST)
18
19     return dst
20
21
22 def pencil_sketch(img):
23     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
24     blr = cv2.GaussianBlur(gray, (0, 0), 3)
25     dst = cv2.divide(gray, blr, scale=255)
26     return dst
```

코드 들여다보기 — 사용한 함수·클래스 (8)

이름	종류	설명
cartoon_filter()	함수	축소 → 양방향 필터 → 에지 합성 → 확대 순으로 카툰 효과 생성
pencil_sketch()	함수	그레이 변환 → 블러 → 나눗셈으로 연필 스케치 효과 생성
img.shape[:2]	속성	영상 배열의 (높이, 너비) 추출
cv2.resize()	함수	영상 크기 변경. 절반 축소로 연산량 감소, 이후 원래 크기로 복원
cv2.INTER_NEAREST	상수	최근접 이웃 보간 — 빠르고 색면이 뭉개지지 않음

cv2.bilateralFilter()	함수	에지는 살리고 색면을 평탄화(카툰 느낌의 핵심)
cv2.Canny()	함수	에지 검출. 임계값 80·120으로 윤곽선 추출
255 - edge	연산	흑백 반전 → 에지가 검은 선, 배경이 흰색
cv2.cvtColor()	함수	1채널 에지를 3채널 BGR로 맞춰 합성 가능하게 함
cv2.bitwise_and()	함수	색면과 반전 에지를 비트 AND → 윤곽선만 검게 남김
cv2.GaussianBlur()	함수	스케치용 흐린 기준 영상 생성
cv2.divide()	함수	원본 ÷ 블러 후 scale=255 곱 → 밝기 대비를 스케치 톤으로 변환

실습 코드 — 카메라 실시간 적용

py

```
1  cap = cv2.VideoCapture(0)
2
3  if not cap.isOpened():
4      print('video open failed!')
5      sys.exit()
6
7  cam_mode = 0
8
9  while True:
10     ret, frame = cap.read()
11
12     if not ret:
13         break
14
15     if cam_mode == 1:
16         frame = cartoon_filter(frame)
17     elif cam_mode == 2:
18         frame = pencil_sketch(frame)
19         frame = cv2.cvtColor(frame, cv2.COLOR_GRAY2BGR)
20
21     cv2.imshow('frame', frame)
22     key = cv2.waitKey(1)
23
24     if key == 27:
25         break
26     elif key == ord(' '):
27         cam_mode += 1
28         if cam_mode == 3:
29             cam_mode = 0
30
31 cap.release()
32 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (9)

이름	종류	설명
cv2.VideoCapture	클래스	카메라·동영상 입력 객체. 인자 0은 기본 웹캠
.isOpened()	메서드	카메라가 정상적으로 열렸는지 확인

<code>.read()</code>	메서드	프레임 1장 읽기 → (성공 여부, 프레임) 튜플 반환
<code>.release()</code>	메서드	카메라 자원 반납(필수)
<code>cv2.waitKey(1)</code>	함수	1ms 대기하며 키 입력 확인 → 실시간 루프 유지
<code>key == 27</code>	조건	ESC 키 아스키 코드 → 루프 종료
<code>ord(' ')</code>	함수	스페이스바 코드 반환 → 모드 순환(0 원본 → 1 카툰 → 2 스케치)
<code>cv2.cvtColor(..., COLOR_GRAY2BGR)</code>	함수	1채널 스케치 결과를 3채널로 되돌려 창 출력 일관성 유지

모드 전환은 스페이스바, 종료는 **ESC**. 카메라가 안 열리면 `cv2.VideoCapture(0)` 의 인덱스를 1, 2로 바꿔 시도할 것.

6. 정리

필터 비교표

필터	함수	주요 인자	특징	적합한 상황
평균값	<code>cv2.blur()</code>	ksize	모든 이웃 동일 가중치, 계산 단순	간단한 흐림, 개념 학습
사용자 정의	<code>cv2.filter2D()</code>	kernel, ddepth	임의 커널 설계 가능	특수 마스크 실험
가우시안	<code>cv2.GaussianBlur()</code>	ksize, sigmaX	거리에 따른 가중 평균, 자연스러운 블러	일반적인 잡음 완화·전처리

메디안	<code>cv2.medianBlur()</code>	ksize(홀수)	중간값 선택(비선형)	소금·후추(임펄스) 잡음
양방향	<code>cv2.bilateralFilter()</code>	d, sigmaColor, sigmaSpace	에지 보존 스무딩, 느림	피부 보정·카툰 효과
샤프닝	<code>GaussianBlur</code> + 연산	sigma, α	2·src - blr로 경계 강조	흐린 영상 선명화

파라미터가 커지면 어떻게 되나

파라미터	커질 때	부작용
평균 필터 ksize	흐림 강해짐	에지 손실, 연산량 증가
가우시안 sigma	흐림 강해짐(커널도 자동 확대)	디테일 손실
메디안 ksize	잡음 제거력 상승	영상이 뭉툭해짐
bilateral sigmaColor	더 넓은 밝기차를 평탄화	에지 보존 효과 감소
bilateral sigmaSpace	참조 범위 확대	속도 급격히 저하
샤프닝 계수 α	선명도 증가	잡음 증폭, 경계 halo

학습 포인트

- 필터링 = 마스크(커널) × 합성곱 — 커널 계수 합이 1이면 밝기 유지
- 평균 → 가우시안: **가중치를 거리 기반으로** 바꾼 것
- 가우시안 → 양방향: **밝기 차이 가중치를** 추가해 에지를 보존한 것
- 선형 필터로 못 잡는 임펄스 잡음은 **메디안(비선형)**으로
- 샤프닝은 **블러를 빼는 것**에서 출발 — 컬러는 YCrCb의 Y 채널만 처리
- 카툰·스케치 같은 응용 효과도 결국 **기본 필터의 조합**



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio